

Requerimientos de Optimización y Ajustes de Lógica – Proyecto CryptolA

Para: Desarrollador de CryptolA

Asunto: Optimizaciones críticas en el script de observación (test_observacion.py), manejo de contexto y lógica de gestión de capital (Money Management).

Hola, Gustavo:

Estuve analizando en detalle el funcionamiento actual del sistema con la nueva arquitectura multi-agente y la actualización a los modelos **Qwen 2.5 de 7B** en las tres instancias. El diseño me parece excelente, pero para garantizar la estabilidad del bot en el largo plazo (24/7) y mejorar la estrategia de gestión de riesgo, necesitamos implementar dos modificaciones prioritarias en el código:

1. Limpieza y Control de Tamaño en los Prompts (Evitar Saturación de Contexto)

Con el cambio de la ventana de noticias a 24 horas y la migración a modelos más grandes (7B), corremos el riesgo de sufrir "parálisis por análisis" o un incremento crítico en los tiempos de inferencia (latencia por ciclo) en días de alta volatilidad.

Aunque la caché MD5 en PostgreSQL funciona muy bien para no repetir noticias pasadas, requerimos un mecanismo de **limpieza y truncado estricto** en el script encargado de armar el prompt antes de enviarlo a Ollama:

- **Límite máximo de caracteres/tokens:** Implementar un tope máximo de noticias o caracteres para el prompt del Agente Fundamental. Si en 24 horas hay una sobrecarga de titulares, el script debe priorizar los más recientes o de mayor impacto, descartando el resto si excede el límite seguro para el modelo 7B.
- **Formateo agresivo de prompts:** Asegurar que tras cada ciclo de 15 minutos, las variables de string que acumulan el texto se limpien por completo de la memoria volátil para evitar fugas de memoria (*memory leaks*) en ejecuciones prolongadas.

2. Rediseño de la Gestión de Capital: Compras Basadas en Importe Inicial (.env)

Actualmente, el sistema opera bajo una lógica "All-In / All-Out", simulando la compra utilizando el 100% del saldo disponible en USDT. Para proteger la cuenta y permitir que el sistema **acumule ganancias de forma real**, debemos modificar la lógica en el script de la siguiente manera:

- **Lectura del archivo .env:** El script debe tomar una variable global (por ejemplo: `IMPORTE_COMPRA_INICIAL=100` o el valor que definamos como base de pruebas).
- **Lógica de Compra con Ganancias Acumuladas:** Si la billetera virtual tiene, por

ejemplo, 150 USDT debido a operaciones ganadoras pasadas, al ejecutarse una nueva señal de COMPRA, el bot **solo debe invertir el importe inicial configurado en el .env (ej: 100 USDT)**. Los 50 USDT restantes quedan resguardados en la cartera como beneficio acumulado.

- **Lógica de Protección ante Pérdidas (Drawdown):** En caso de que hayamos tenido operaciones en pérdida (Stop-Loss) y el saldo total de la cartera en USDT caiga **por debajo** del importe inicial configurado (ejemplo: la cartera bajó a 85 USDT y el .env exige 100), el bot deberá ejecutar la compra utilizando **el 100% del saldo disponible** en ese momento (85 USDT).

Quedo a la espera de tus comentarios para ver qué tan viable es empaquetar estos cambios en el próximo push/actualización del repositorio dentro de la carpeta tests.

¡Muchas gracias por el gran trabajo que vienes haciendo con la arquitectura!